UNIVERSIDADE DE MOGI DAS CRUZES

CURSO DE SISTEMA DE INFORMAÇÃO

LUCAS MARQUIOL

SISTEMA DE LOGIN SEGURO

Documentação técnica

Mogi das Cruzes
2026

LUCAS MARQUIOL

SISTEMA DE LOGIN SEGURO

Documentação técnica apresentada à disciplina Aplicativos Web, do curso de Sistema de Informação da Universidade de Mogi das Cruzes, como parte da atividade de desenvolvimento de um sistema de Login Seguro.

Professor: Alessandro Aparecido da Silva Horas.

Mogi das Cruzes
2026

SUMÁRIO

1 INTRODUÇÃO 3

2 ESTRUTURA DO SISTEMA 4

3 AUTENTICAÇÃO E AUTORIZAÇÃO 5

4 INTEGRAÇÃO COM O MONGODB ATLAS 6

5 ARMAZENAMENTO DAS SESSÕES 7

6 INTERFACE E PERSONALIZAÇÃO DO TEMA 8

7 CONFIGURAÇÃO E EXECUÇÃO LOCAL 9

8 VERSIONAMENTO E ENTREGA 10

9 VERIFICAÇÕES FUNCIONAIS 11

10 CONCLUSÃO 12

REFERÊNCIAS 13

1 INTRODUÇÃO

Este documento apresenta a estrutura e o funcionamento de um sistema de Login Seguro desenvolvido com Java, Spring Boot, Thymeleaf, Spring Security e MongoDB Atlas. A aplicação permite cadastrar contas, autenticar usuários, encerrar sessões e controlar o acesso às páginas conforme o perfil atribuído a cada conta.

A documentação tem como finalidade explicar a organização do código, a integração com o banco de dados, as decisões de implementação e o procedimento de execução local. Essas informações permitem compreender a solução e adaptar sua apresentação visual e seus módulos ao tema de um futuro Projeto Final de Curso, preservando o núcleo de autenticação.

1.1 Objetivo e escopo

O objetivo é implementar o controle de usuários com validação de dados, senhas armazenadas como hash e autorização por três perfis: USUARIO, MODERADOR e ADMIN. O administrador pode listar usuários, editar nome e e-mail, alterar perfis e excluir contas. O cadastro público cria somente contas com o perfil USUARIO.

O escopo corresponde à atividade acadêmica. A aplicação possui páginas de cadastro, login, início, painel do moderador, listagem administrativa e edição de usuários. O armazenamento contempla os dados das contas e as sessões HTTP no MongoDB Atlas. A interface utiliza fragments compartilhados e um tema CSS selecionado por configuração.

1.2 Ambiente de desenvolvimento

O projeto utiliza Java 21, Spring Boot 4.0.8 e Maven Wrapper. O desenvolvimento foi realizado no Visual Studio Code, com extensões para Java e Spring Boot. A integração de sessões utiliza a dependência org.mongodb:mongodb-spring-session:4.0.0. Git e branches organizadas segundo Gitflow são utilizados para registrar e integrar as etapas de desenvolvimento.

2 ESTRUTURA DO SISTEMA

O pacote principal é com.lucas.login_seguro. A organização separa as responsabilidades da aplicação, evitando concentrar acesso ao banco, regras de cadastro e apresentação no mesmo arquivo. A classe LoginSeguroApplication inicia a aplicação e fica acima dos demais pacotes na hierarquia.

Pacote	Responsabilidade
config	PasswordConfig, SecurityConfig, SessionConfig e TemaConfig configuram hash, rotas, sessões e tema.
controller	AuthController, PaginaController e UsuarioController recebem requisições e selecionam templates.
dto	CadastroUsuarioDto e EdicaoUsuarioDto representam e validam os dados dos formulários.
model	Usuario representa uma conta; Role define os três perfis.
repository	UsuarioRepository consulta e persiste contas com Spring Data MongoDB.
service	UsuarioService aplica regras; UsuarioDetailsService fornece as contas ao Spring Security.

2.1 Modelo de usuário e acesso aos dados

Usuario contém id, nome, email, senha e role. A anotação @Document associa a classe à coleção usuarios_login_seguro. O identificador é representado por String e marcado com @Id. O campo senha recebe o hash BCrypt, e role contém um valor do enum Role.

UsuarioRepository estende MongoRepository<Usuario, String>. Os métodos findByEmail e existsByEmail permitem localizar uma conta e verificar a existência de seu e-mail. As operações de salvar, listar, buscar por ID e excluir são fornecidas pelo repositório. O índice único do e-mail também protege o banco contra gravações concorrentes duplicadas.

2.2 Fluxo entre as camadas

O formulário envia os dados para o controller, que aplica a validação do DTO e chama o serviço. O serviço verifica as regras e utiliza o repositório. Em caso de erro, o controller retorna o formulário com a mensagem correspondente. Em caso de sucesso, redireciona para a página apropriada. Os templates permanecem responsáveis pela apresentação.

3 AUTENTICAÇÃO E AUTORIZAÇÃO

3.1 Cadastro e hash de senha

O cadastro exige nome, e-mail, senha e confirmação da senha. As validações incluem campos obrigatórios, formato de e-mail, nome com até 100 caracteres e e-mail com até 254 caracteres. A senha deve ter pelo menos oito caracteres; o serviço também verifica o limite de 72 bytes em UTF-8 usado pelo BCrypt e exige igualdade com a confirmação.

O serviço remove espaços externos do nome e do e-mail e converte o e-mail para letras minúsculas com Locale.ROOT. Antes de salvar, verifica duplicidade, gera o hash com PasswordEncoder e atribui Role.USUARIO. PasswordConfig fornece BCryptPasswordEncoder. Hash é uma transformação de mão única: a autenticação compara a senha digitada com o hash, sem recuperar a senha original (Spring Security, [s. d.]).

3.2 Login e logout

GET /login exibe o template. POST /login é processado pelo Spring Security, com os parâmetros email e senha. UsuarioDetailsService busca o e-mail normalizado no MongoDB e fornece o hash e as autoridades da conta. O sucesso redireciona para /inicio; uma falha exibe uma mensagem genérica de credenciais inválidas.

O logout usa POST /logout, invalida a sessão e remove os cookies configurados JSESSIONID e SESSION. Os formulários usam th:action e a proteção CSRF permanece habilitada. Não existe endpoint próprio no controller para comparar senhas ou autenticar manualmente.

3.3 Perfis e rotas

Rota ou operação	Permissão
/cadastro, /login, /css/** e /images/**	Pública
/inicio	Qualquer conta autenticada
/moderador/**	MODERADOR ou ADMIN
/admin/**	Somente ADMIN
Demais rotas	Conta autenticada

As restrições são aplicadas no servidor por SecurityFilterChain. A conta pública não escolhe seu perfil. O administrador modifica perfis por um formulário protegido na área administrativa.

4 INTEGRAÇÃO COM O MONGODB ATLAS

MongoDB Atlas mantém o banco login_seguro. A aplicação utiliza a coleção usuarios_login_seguro para armazenar as contas. Esse nome separa a atividade de documentos de outra implementação que já existiam na coleção usuarios, sem excluir os dados anteriores.

4.1 Preparação do acesso

No Atlas, é necessário criar ou utilizar um cluster, cadastrar um usuário de banco e autorizar o IP do computador na IP Access List. O usuário login_app recebe readWrite no banco login_seguro. Essa credencial pertence à aplicação e não corresponde às contas que entram pela página de login.

A URI é obtida em Connect, na opção de drivers para Java. O formato mongodb+srv utiliza a descoberta DNS do cluster e habilita TLS por padrão. A conexão preserva a validação do certificado; não se configura a aceitação de certificados inválidos (MongoDB, [s. d.]a). Credenciais com caracteres reservados precisam de codificação apropriada para URI.

4.2 Arquivo de configuração

```
spring.application.name=login-seguro
spring.mongodb.uri=${MONGODB_URI}
spring.mongodb.database=login_seguro
spring.data.mongodb.auto-index-creation=true
app.tema=padrao
```

O arquivo é src/main/resources/application.properties. No Spring Boot 4.0, spring.mongodb.uri define a URI e spring.mongodb.database seleciona o banco. A propriedade auto-index-creation permanece no namespace spring.data.mongodb (Spring Boot, [s. d.]). A variável MONGODB_URI contém a conexão real e é definida fora do código versionado.

4.3 Integridade dos e-mails

A anotação @Indexed(unique = true) no campo email e a criação automática de índices habilitam a restrição de unicidade. O serviço verifica duplicidades antes da gravação e trata DuplicateKeyException. A mesma regra é usada na edição, permitindo manter o e-mail da própria conta, mas impedindo usar o e-mail de outra.

Dados duplicados previamente existentes impedem a criação do índice. A solução adotada foi utilizar a coleção própria da atividade, mantendo a restrição de integridade e preservando a coleção anterior.

5 ARMAZENAMENTO DAS SESSÕES

As sessões HTTP também são armazenadas no MongoDB Atlas, atendendo ao requisito de persistência além das contas. A extensão Spring Session MongoDB substitui o armazenamento de sessão associado apenas ao processo da aplicação por um repositório no banco (MongoDB, [s. d.]b).

5.1 Dependência e configuração

```
<dependency>
<groupId>org.mongodb</groupId>
<artifactId>mongodb-spring-session</artifactId>
<version>4.0.0</version>
</dependency>
```

O bloco acima integra a lista dependencies do pom.xml. SessionConfig está no pacote config e utiliza a configuração abaixo:

```
@Configuration
@EnableMongoHttpSession(maxInactiveIntervalInSeconds = 1800)
public class SessionConfig {
}
```

A anotação pertence ao pacote org.mongodb.spring.session.config.annotation.web.http. A integração utiliza o acesso MongoDB configurado na aplicação. A coleção sessions contém os documentos das sessões e deve ser verificada no banco login_seguro após o login.

5.2 Prazo e persistência

A sessão expira após 1.800 segundos, equivalentes a 30 minutos sem atividade. Esse prazo é uma decisão do projeto. Se a aplicação reiniciar, a sessão válida pode ser recuperada do banco quando o navegador conserva seu cookie. Não é necessário criar uma coleção manual de tokens ou implementar um login por JWT.

5.3 Encerramento e alterações de conta

O logout invalida a sessão usada pela conta. Foi verificado que, após sair, acessar /inicio volta a exigir autenticação. A expiração por inatividade foi configurada, mas não foi medida em um teste de espera de 30 minutos.

As autoridades são carregadas no login. Alterar um perfil não atualiza automaticamente sessões já abertas; é necessário sair e entrar novamente para carregar as novas permissões. A exclusão de uma conta também não implementa revogação automática de todas as sessões que essa conta possa ter aberto anteriormente.

6 INTERFACE E PERSONALIZAÇÃO DO TEMA

Thymeleaf renderiza os templates em src/main/resources/templates. O CSS fica em src/main/resources/static/css/temas. Essa separação permite alterar a apresentação sem modificar UsuarioService, UsuarioRepository ou a configuração de autenticação.

Template	Finalidade
cadastro.html	Cadastro público e mensagens de validação.
login.html	Autenticação e mensagens de cadastro, erro e logout.
inicio.html	Identificação da conta conectada e logout.
moderador/painel.html	Página protegida para MODERADOR e ADMIN.
admin/usuarios.html	Listagem, links de edição e formulários de exclusão.
admin/editar-usuario.html	Edição de nome, e-mail e perfil.

6.1 Fragments compartilhados

A pasta fragments contém head.html, header.html e footer.html. O primeiro centraliza os metadados, o título recebido por parâmetro e o carregamento do CSS. Os outros dois compartilham o cabeçalho e o rodapé. As páginas inserem esses elementos com th:replace.

```
<head th:replace="~{fragments/head :: head('Login')}"></head>
<header th:replace="~{fragments/header :: header}"></header>
<footer th:replace="~{fragments/footer :: footer}"></footer>
```

6.2 Tema configurável

TemaConfig utiliza @ControllerAdvice e fornece o atributo tema aos modelos. A propriedade app.tema=padrao seleciona padrao.css. O fragment head resolve o caminho /css/temas/{nome}.css a partir desse valor.

Para personalizar, cria-se um CSS na mesma pasta, por exemplo personalizado.css, muda-se a propriedade para app.tema=personalizado e reinicia-se a aplicação. O teste realizado criou temporariamente teste.css, alterou o fundo e confirmou a seleção pela configuração. Ao final, o tema voltou para padrao e o arquivo de teste foi removido.

7 CONFIGURAÇÃO E EXECUÇÃO LOCAL

7.1 Preparação do ambiente

É necessário instalar JDK 21 e Git, obter o código-fonte e abrir a pasta principal que contém pom.xml e mvnw.cmd. Maven Wrapper baixa a versão necessária do Maven e as dependências; não é exigida uma instalação separada de Maven. A primeira execução depende de acesso à internet.

No Atlas, prepare o cluster, o usuário com readWrite no banco login_seguro e o IP autorizado. Copie a URI real do cluster. As credenciais devem ser mantidas fora do repositório público.

7.2 Execução no PowerShell

No terminal aberto na pasta principal do projeto, defina a variável de ambiente. O exemplo apresenta marcadores que devem ser substituídos pelos valores reais:

```
$env:MONGODB_URI = 'mongodb+srv://USUARIO:SENHA@SEU_CLUSTER/'
```

A URI deve ficar entre aspas simples. Preserve os parâmetros fornecidos pelo Atlas quando utilizar sua URI. Se houver caracteres reservados nas credenciais, codifique-os antes da inclusão. Não use a senha de uma conta do sistema no lugar da senha do usuário do banco.

```
Test-Path Env:MONGODB_URI
.\mvnw.cmd spring-boot:run
```

O primeiro comando deve retornar True. O segundo inicia a aplicação. Acesse http://localhost:8080/login ou /cadastro. Para parar, pressione Ctrl + C. Ao abrir outro terminal ou reiniciar o computador, configure MONGODB_URI novamente, pois a atribuição apresentada vale para a sessão atual do PowerShell.

7.3 Primeiro administrador

Cadastre uma conta pelo formulário público. No Atlas, localize essa conta em login_seguro, coleção usuarios_login_seguro, e altere somente role de USUARIO para ADMIN, mantendo o tipo String. Faça logout se a conta estiver conectada e entre novamente. A administração fica em /admin/usuarios. A partir daí, alterações de perfil dos demais usuários são feitas pela interface.

7.4 Diagnóstico básico

Se a aplicação encerrar, consulte a causa anterior à mensagem final do Maven. Verifique a variável, as credenciais, o IP autorizado e eventuais duplicidades de índice. Um erro 403 em uma rota protegida indica falta de permissão. Não publique logs que exibam credenciais ou conteúdo de sessões.

8 VERSIONAMENTO E ENTREGA

O versionamento segue Gitflow. main representa a versão de entrega, develop integra funcionalidades e feature/* isola cada etapa. O desenvolvimento utilizou branches para configuração do Atlas, cadastro e autenticação, sessões no MongoDB, controle de usuários, templates e tema e documentação.

Os commits registram alterações por finalidade e as integrações utilizam merge --no-ff. Uma cópia inicial duplicada do projeto foi removida do repositório, mantendo a implementação principal. A documentação é desenvolvida em feature/documentacao.

8.1 Preparação da versão de entrega

Depois de revisar o README e adicionar o PDF em docs/documentacao.pdf, registre os arquivos e integre feature/documentacao em develop. Para preparar a entrega, crie uma branch release/1.0.0 a partir de develop, revise a execução e então integre a release em main e develop. A tag v1.0.0 identifica a versão entregue.

```
git switch develop
git switch -c release/1.0.0
# Após a revisão e os commits da release:
git switch main
git merge --no-ff release/1.0.0
git tag -a v1.0.0 -m "Entrega da atividade Login Seguro"
git switch develop
git merge --no-ff release/1.0.0
```

8.2 Publicação no GitHub pessoal

Crie um repositório público separado do repositório do PFC. Configure origin com a URL real desse repositório e envie main, develop e as tags. A URL deve ser informada na entrega da atividade. Os comandos abaixo utilizam um marcador para a URL:

```
git remote add origin URL_DO_REPOSITORIO
git push -u origin main
git push -u origin develop
git push origin --tags
```

O repositório deve incluir código-fonte, pom.xml, Maven Wrapper, application.properties sem credenciais, README e PDF. O README explica ambiente, configuração do Atlas, execução, primeiro administrador, perfis, estrutura e personalização. Pastas target e arquivos locais com segredos devem ficar fora do versionamento. hotfix/* é reservado para correções urgentes de uma versão já entregue, quando necessário.

9 VERIFICAÇÕES FUNCIONAIS

Os resultados abaixo correspondem às verificações manuais realizadas durante a construção e confirmadas pelo desenvolvedor. Eles registram comportamentos observados; não representam uma auditoria de segurança ou uma suíte de testes automatizados.

Verificação	Resultado observado
Cadastro válido	Conta criada e redirecionamento para login.
Senhas diferentes no cadastro	Cadastro rejeitado com mensagem.
E-mail já cadastrado	Cadastro rejeitado com mensagem.
Login válido e inválido	Entrada permitida com credenciais corretas; falha com senha errada.
Hash e índice no Atlas	Hash BCrypt, perfil USUARIO e índice único de e-mail confirmados.
USUARIO na administração	Acesso negado com status 403.
ADMIN na administração	Listagem de usuários exibida.
MODERADOR nas áreas protegidas	Painel permitido; administração bloqueada.
Edição de nome e perfil	Dados atualizados e perfil MODERADOR aplicado após novo login.
E-mail duplicado na edição	Alteração rejeitada sem trocar a conta.
Exclusão de conta de teste	Conta removida da tabela e da coleção.
Persistência de sessão	Coleção sessions verificada; login mantido após reiniciar a aplicação.
Logout	Acesso posterior a /inicio redirecionou ao login.
Tema e fragments	CSS aplicado, elementos compartilhados exibidos e troca de tema confirmada.

A expiração da sessão e os limites de tamanho dos campos estão definidos no código, mas não foram confirmados por testes específicos de duração ou de todos os valores de fronteira. Esses casos não são apresentados como resultados executados.

10 CONCLUSÃO

A implementação reúne cadastro validado, autenticação por e-mail, armazenamento de senhas com BCrypt, logout e autorização por três perfis. A administração permite listar, editar e excluir usuários e alterar seus perfis. As contas e as sessões são persistidas no MongoDB Atlas, com configuração da conexão por variável de ambiente.

A separação entre controllers, DTOs, serviços, repositório, modelos e configurações facilita localizar responsabilidades e manter a lógica central. Na interface, os fragments e o tema CSS configurável permitem alterar a apresentação de modo independente das regras de cadastro e autenticação.

10.1 Orientações para adaptação ao PFC

Uma adaptação visual pode começar pelos arquivos CSS, pelo cabeçalho e pelo rodapé. O nome configurado em app.tema seleciona o estilo. Para incluir funcionalidades do tema do PFC, podem ser criados novos modelos, DTOs, serviços, repositórios, controllers e páginas seguindo a mesma divisão de responsabilidades.

As novas rotas devem receber regras explícitas de acesso em SecurityConfig quando exigirem permissões específicas. A autenticação pode continuar usando UsuarioDetailsService e PasswordConfig. Alterações no modelo de usuário devem considerar os documentos e índices já existentes no banco; não é necessário substituir o núcleo apenas para mudar o tema visual.

10.2 Comportamentos e limites da versão

A versão documentada utiliza HTTP no ambiente local em localhost. TLS protege a conexão com o Atlas, mas não transforma automaticamente a conexão entre navegador e aplicação em HTTPS. A aplicação mantém o hash da senha ao editar nome, e-mail e perfil; não foi criada uma função de troca de senha.

Mudanças de e-mail e perfil exigem novo login para atualizar a identidade ou as permissões da sessão. A exclusão remove o documento da conta, sem uma rotina de revogação de todas as sessões anteriores. Esses comportamentos descrevem a implementação atual e devem ser considerados em futuras adaptações.

A entrega reúne o código e o README no repositório público e esta documentação em PDF. A publicação no GitHub e o fechamento da release são procedimentos de entrega descritos neste documento, a serem concluídos após a revisão final dos arquivos.

REFERÊNCIAS

MONGODB. Connection strings. [S. l.], [s. d.]a. Disponível em: https://www.mongodb.com/docs/manual/reference/connection-string/. Acesso em: 8 out. 2026.

MONGODB. Spring Session MongoDB integration. [S. l.], [s. d.]b. Disponível em: https://www.mongodb.com/docs/drivers/java/sync/current/integrations/spring-session/. Acesso em: 8 out. 2026.

MONGODB. MongoDB Spring Session extension. [S. l.], [s. d.]c. Disponível em: https://github.com/mongodb/mongo-spring-session. Acesso em: 8 out. 2026.

SPRING BOOT. Common application properties. Versão 4.0. [S. l.], [s. d.]. Disponível em: https://docs.spring.io/spring-boot/4.0/appendix/application-properties/index.html. Acesso em: 8 out. 2026.

SPRING SECURITY. Password storage. [S. l.], [s. d.]. Disponível em: https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html. Acesso em: 8 out. 2026.

UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO. Normalização. Sistema Integrado de Bibliotecas. Vitória: UFES, [s. d.]. Disponível em: https://biblioteca.ufes.br/normalizacao. Acesso em: 8 out. 2026.